



Instituto Federal do Ceará - Embarcotech

Relatório Final do Projeto

**Sistema IoT para Medição e Análise de Consumo de
Energia Elétrica**

Autor: José Adriano Filho

Matrícula: 2025101109806

Jan/2025

Sumário

1 INTRODUÇÃO	3
2 OBJETIVOS	4
2.2 Objetivos Específicos	4
3 JUSTIFICATIVA.....	4
4 PROBLEMÁTICA.....	4
5 ARQUITETURA DO SISTEMA	5
5.1 Arquitetura de Hardware.....	5
5.2 Arquitetura de Firmware.....	7
5.3 Arquitetura IoT	10
6 DESENVOLVIMENTO DO SISTEMA	11
7 EVIDÊNCIAS DE FUNCIONAMENTO	14
8 CONCLUSÃO	14
9 REFERÊNCIAS	15
10 APÊNDICES	16

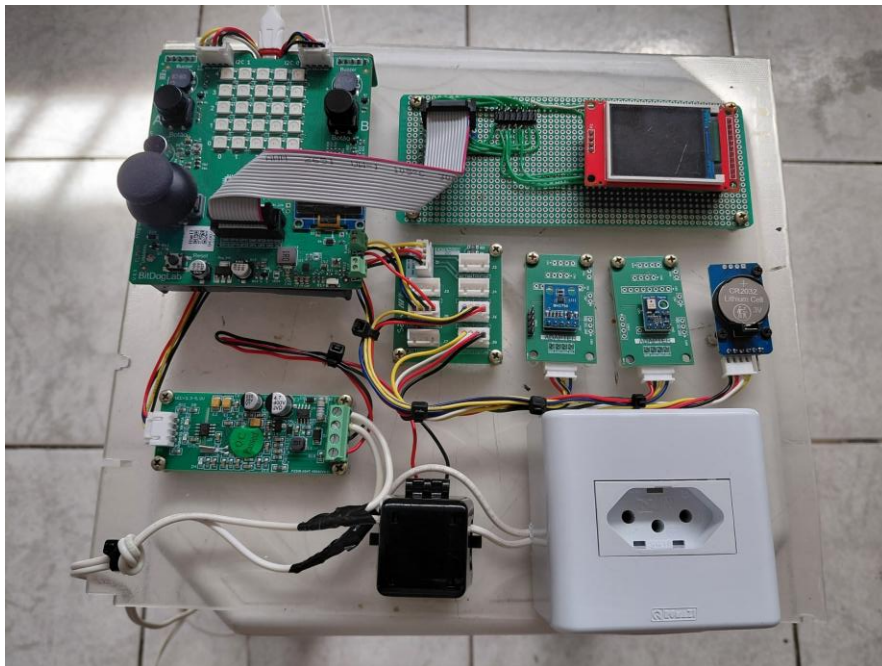
SISTEMA EMBARCADO PARA MONITORAMENTO DE ENERGIA

1 INTRODUÇÃO

A crescente evolução das tecnologias associadas à Internet das Coisas (IoT) tem possibilitado o desenvolvimento de sistemas embarcados capazes de realizar o monitoramento remoto de grandezas físicas de forma contínua, segura e eficiente. Esses sistemas vêm sendo amplamente aplicados em contextos residenciais, industriais e educacionais, permitindo a coleta de dados, o processamento local e o envio das informações para plataformas em nuvem.

No contexto dos sistemas embarcados modernos, a utilização de sistemas operacionais de tempo real (RTOS) torna-se essencial para o gerenciamento de múltiplas tarefas concorrentes, garantindo previsibilidade temporal, modularidade e escalabilidade do firmware. Associado a isso, a comunicação segura de dados é um requisito fundamental, especialmente em aplicações IoT que envolvem redes públicas.

Dessa forma, este trabalho apresenta o desenvolvimento de um sistema embarcado baseado na Raspberry Pi Pico W, utilizando o sistema operacional FreeRTOS, sensores ambientais e de energia elétrica, além de comunicação via protocolo MQTT. O sistema desenvolvido permite a aquisição, visualização local e transmissão remota dos dados, proporcionando uma solução completa de monitoramento.



2 OBJETIVOS

Desenvolver um sistema embarcado com conectividade Wi-Fi capaz de monitorar variáveis ambientais e grandezas elétricas, utilizando arquitetura multitarefa e comunicação segura com plataforma IoT.

2.2 Objetivos Específicos

- Implementar firmware em linguagem C utilizando o SDK oficial da Raspberry Pi Pico W;
- Utilizar o sistema operacional FreeRTOS para gerenciamento das tarefas;
- Integrar sensores ambientais de temperatura, umidade e luminosidade;
- Implementar a leitura de grandezas elétricas por meio de protocolo Modbus RTU;
- Exibir informações em tempo real em um display gráfico;
- Estabelecer comunicação Wi-Fi estável;
- Implementar comunicação MQTT;
- Validar o funcionamento do sistema por meio de testes práticos.

3 JUSTIFICATIVA

O monitoramento de energia elétrica é uma ferramenta essencial para a identificação de desperdícios, análise de consumo e melhoria da eficiência energética. Quando associado ao monitoramento de variáveis ambientais, possibilita uma compreensão mais abrangente do comportamento do sistema analisado.

Sob o ponto de vista acadêmico, o projeto se justifica pela integração prática de conceitos fundamentais da área de sistemas embarcados e IoT, tais como sistemas operacionais de tempo real, comunicação serial, protocolos de rede, segurança da informação e integração hardware e software.

Além disso, a solução desenvolvida apresenta potencial de aplicação em ambientes reais, podendo ser expandida para diferentes contextos, o que reforça sua relevância técnica e prática.

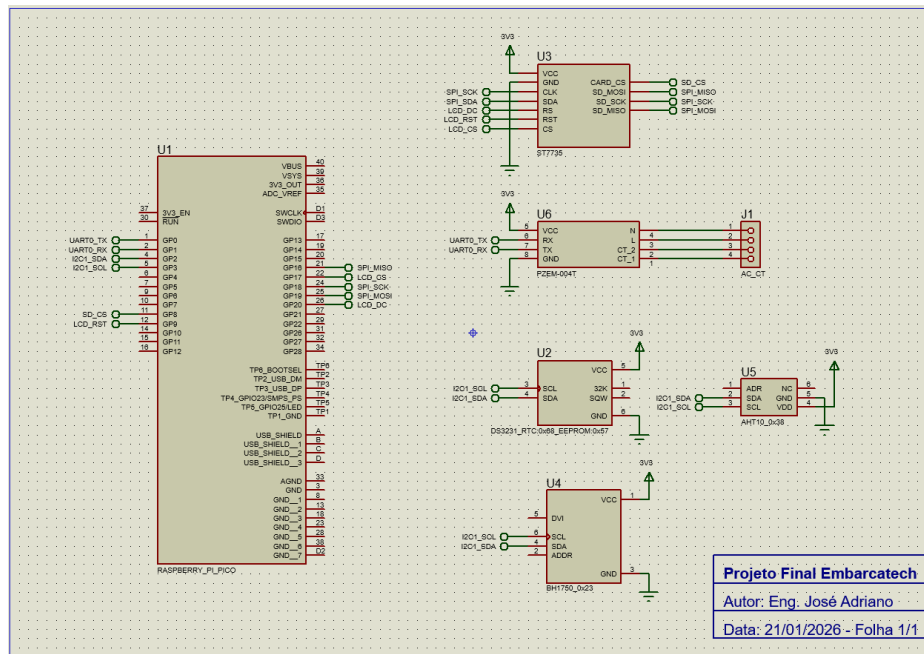
4 PROBLEMÁTICA

A maioria das residências não dispõe de um sistema acessível e inteligente que permita monitorar, em tempo real, o consumo elétrico e as condições ambientais. Isso dificulta a identificação de desperdícios e o controle eficiente do uso de energia. Além disso, os medidores convencionais não fornecem dados históricos detalhados nem alertas automatizados sobre consumo excessivo ou falhas no fornecimento.

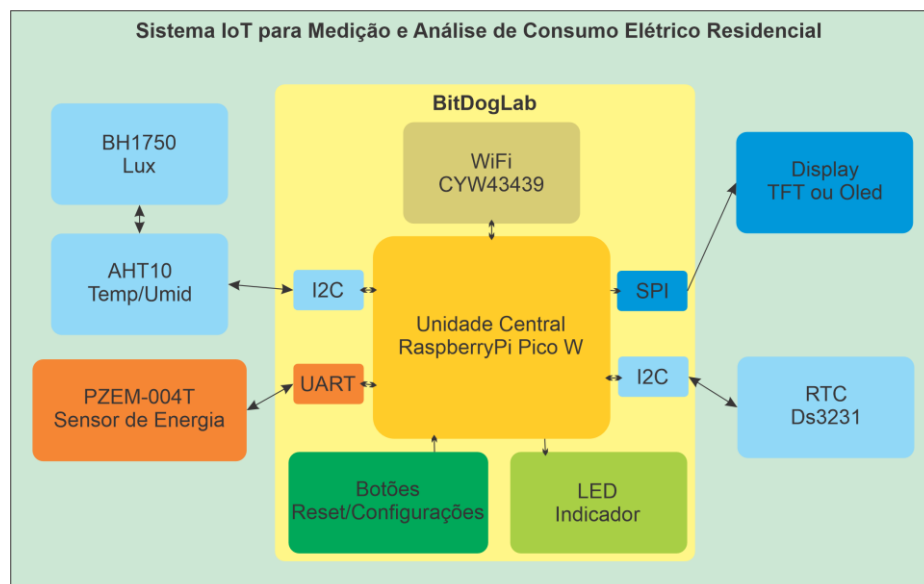
5 ARQUITETURA DO SISTEMA

5.1 Arquitetura de Hardware

O sistema de hardware é composto por uma Raspberry Pi Pico W como unidade central de processamento, sensores ambientais para medição de temperatura, umidade e luminosidade, um módulo de medição de energia elétrica, um display gráfico para interface local, além de módulos auxiliares como RTC, botões e LEDs.



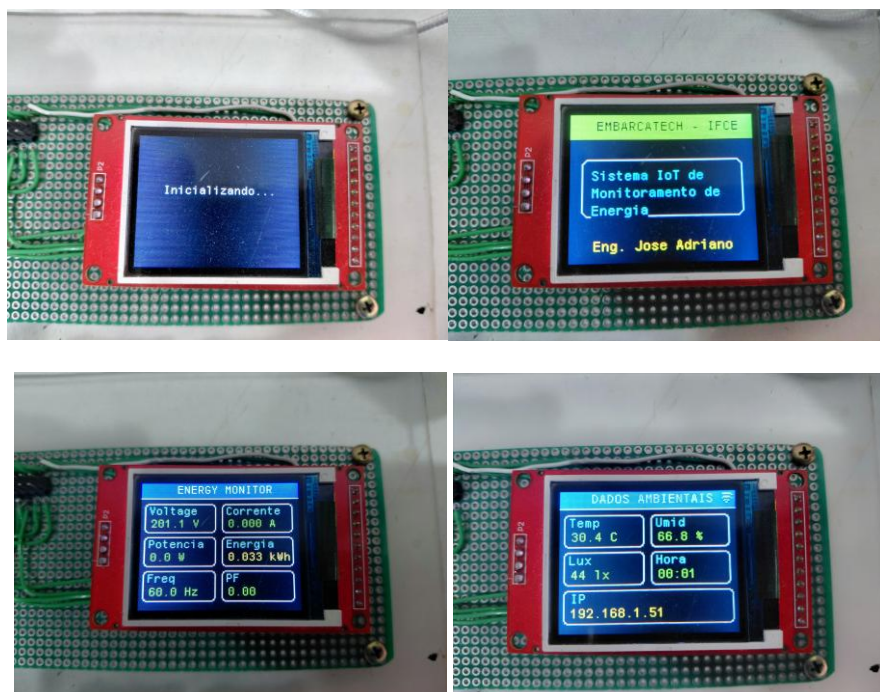
A comunicação entre os sensores ambientais e o microcontrolador ocorre por meio do barramento I2C, enquanto o módulo de energia utiliza comunicação UART com protocolo Modbus RTU. O display gráfico é conectado via SPI.



PINOS DE CONTROLE PROJETO FINAL - EmbarcaTech	
Sistema IoT para Medição e Análise de Consumo Elétrico Residencial	
Descrição	GPIO
UART_TX - Pino de transmissão para o módulo sensor de energia	GPIO_0
UART_RX - Pino de recepção para o módulo sensor de energia	GPIO_1
I2C1_SDA - Pino de dados da I2C para os sensores, RTC e memória	GPIO_2
I2C1_SCL - Pino de clock da I2C para os sensores, RTC e memória	GPIO_3
SD_CS - Pino de chip select do cartão SD no LCD	GPIO_8
LCD_RST - Pino de reset do LCD ST7735	GPIO_9
SPI_MISO - Pino de Master Input Slave Output do LCD ST7735	GPIO_16
LCD_CS - Pino de chip select do LCD ST7735	GPIO_17
SPI_SCK - Pino de clock da interface SPI do LCD ST7735	GPIO_18
SPI_MOSI - Pino de Master Output Slave Input do LCD ST7735	GPIO_19
LCD_DC (RS) - Pino de controle do LCD ST7735	GPIO_20

Durante o desenvolvimento do sistema, foram identificados desafios técnicos relacionados à escolha dos sensores, à comunicação segura e à integração multitarefa. Inicialmente, foi utilizado um módulo de medição de energia baseado no circuito integrado HLW8032, que apresentou dificuldades relacionadas à calibração, instabilidade nas leituras e maior complexidade de implementação.

Como solução, optou-se pela substituição do módulo HLW8032 pelo PZEM-004T versão 4.0, que utiliza protocolo Modbus RTU e apresentou maior confiabilidade e facilidade de integração.



5.2 Arquitetura de Firmware

Link: https://github.com/EngAdriano/energy_monitoring

O firmware foi estruturado seguindo uma abordagem modular, na qual cada funcionalidade do sistema é encapsulada em módulos independentes. Essa organização permite maior facilidade de manutenção, reutilização de código e expansão futura do sistema.

Módulo	Arquivo / Diretório	Responsabilidade
Inicialização do Sistema	main.c	Configuração do hardware, inicialização do FreeRTOS, criação das tarefas e início do escalonador.
Gerenciamento de Estado	system_state.c/h	Armazena e fornece acesso controlado ao estado global do sistema, incluindo dados ambientais e elétricos.
Sensores Ambientais	lib/sensors/	Implementa drivers e rotinas de leitura dos sensores de temperatura, umidade e luminosidade via I2C.
Medição de Energia	lib/pzem004t/	Implementa a comunicação Modbus RTU com o módulo PZEM-004T para aquisição de grandezas elétricas.
Display Gráfico	lib/display/	Controla a interface gráfica local, realizando a renderização das informações no display.
Comunicação MQTT	lib/mqtt_manager/	Gerencia a conexão Wi-Fi, serialização dos dados e publicação no broker MQTT.
Servidor Web	webserver_task.c/h	Implementa o servidor HTTP embarcado, páginas HTML, autenticação, controle de sessão e endpoints REST-like.
Autenticação e Segurança	auth_storage.c/h	Gerencia credenciais de acesso, realizando hash SHA-256 e armazenamento seguro na EEPROM do RTC.
Criptografia	sha256.c/h	Implementa o algoritmo criptográfico SHA-256 utilizado para proteção das senhas.
RTC e EEPROM	rtc_service.c/h	Controla data e hora do sistema e armazenamento persistente de dados críticos na EEPROM do módulo RTC.

A inicialização do sistema é realizada no arquivo principal da aplicação, responsável pela configuração do hardware, inicialização do sistema operacional FreeRTOS e criação das tarefas. Após a inicialização, o controle do fluxo de execução passa a ser gerenciado exclusivamente pelo escalonador do FreeRTOS.

Tarefa	Função Principal	Prioridade	Periodicidade / Evento	Descrição Técnica
TaskSensores	Aquisição de dados ambientais	Média	Periódica	Realiza a leitura dos sensores ambientais (temperatura, umidade e luminosidade) por meio do barramento I2C. Atualiza a estrutura

				global de estado do sistema com os valores mais recentes.
TaskEnergia	Medição de grandezas elétricas	Média	Periódica	Comunica-se com o módulo PZEM-004T via UART utilizando protocolo Modbus RTU, realizando a leitura de tensão, corrente, potência, energia, frequência e fator de potência.
TaskDisplay	Interface local	Baixa	Periódica	Atualiza o display gráfico com os dados de energia e variáveis ambientais, apresentando as informações ao usuário em tempo real.
TaskMQTT	Comunicação IoT	Alta	Periódica / Evento	Gerencia a conexão Wi-Fi e a comunicação MQTT, publicando periodicamente os dados do sistema no broker IoT. Implementa lógica básica de reconexão.
TaskWebServer	Interface web embarcada	Média	Evento (requisições TCP)	Implementa um servidor HTTP utilizando lwIP RAW TCP, fornecendo interface web para monitoramento e configuração do sistema com controle de autenticação e sessão.
TaskUI	Gerenciamento de estados	Baixa	Evento	Monitora botões físicos e eventos internos, realizando transições de estado do sistema e interações locais do usuário.

Cada tarefa do sistema possui responsabilidade bem definida, sendo executada de forma concorrente e cooperativa. A comunicação entre tarefas é realizada por meio de filas (queues), evitando o compartilhamento direto de variáveis globais e garantindo maior integridade e sincronização dos dados.

Essa arquitetura permite que o sistema execute simultaneamente a leitura de sensores, atualização da interface gráfica e transmissão de dados via MQTT, sem comprometer a estabilidade ou previsibilidade temporal do firmware.

Além das tarefas responsáveis pela aquisição de dados e comunicação MQTT, o sistema implementa uma interface de configuração baseada em servidor web embarcado. Essa interface foi desenvolvida utilizando a pilha lwIP em modo RAW TCP, permitindo a criação de um servidor HTTP leve e adequado às restrições de memória do microcontrolador.

O servidor web executa como uma tarefa dedicada do FreeRTOS, sendo responsável por atender requisições HTTP, disponibilizar páginas HTML estáticas e fornecer dados dinâmicos no formato JSON. Por meio dessa interface, o usuário pode visualizar em tempo real as grandezas elétricas e ambientais, além de realizar configurações do sistema, como ajuste de data e hora do RTC e gerenciamento das credenciais de acesso.

Para garantir a segurança do acesso, o servidor web implementa um mecanismo de autenticação baseado em usuário e senha, associado a um controle de sessão com tempo de

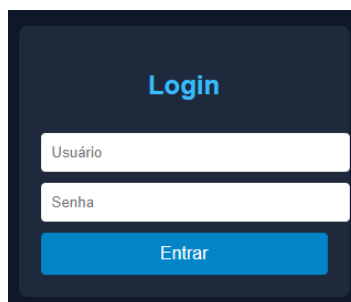
expiração. Dessa forma, somente usuários autenticados podem acessar as páginas de configuração e os endpoints de dados sensíveis.

Endpoint	Método HTTP	Autenticação	Descrição Funcional
/	GET	Condicional	Página principal do sistema. Caso o usuário esteja autenticado, exibe o painel de monitoramento (dashboard). Caso contrário, redireciona para a página de login.
/login	POST	Não	Endpoint responsável pelo processo de autenticação. Recebe usuário e senha, gera o hash SHA-256 da senha e compara com o valor armazenado na EEPROM do RTC.
/logout	GET	Sim	Finaliza a sessão ativa do usuário, invalidando o controle de sessão e redirecionando para a página de login.
/config/auth	GET	Sim	Exibe a interface web para configuração das credenciais de acesso (usuário e senha).
/config/auth	POST	Sim	Recebe novas credenciais de acesso. A senha é processada por hash SHA-256 e armazenada de forma persistente na EEPROM do módulo RTC.
/config/time	GET	Sim	Exibe a interface web para configuração manual da data e hora do sistema.
/config/time	POST	Sim	Recebe data e hora fornecidas pelo usuário, valida os valores e atualiza o RTC do sistema.
/energy	GET	Sim	Retorna dados de medição elétrica em formato JSON, incluindo tensão, corrente, potência, energia, frequência e fator de potência.
/env	GET	Sim	Retorna dados ambientais em formato JSON, incluindo temperatura, umidade e luminosidade.

Nota:

Todos os endpoints protegidos exigem uma sessão válida. O sistema utiliza controle de sessão com tempo de expiração baseado em inatividade, aumentando a segurança contra acessos não autorizados.

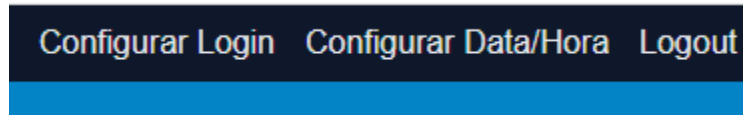
Abaixo podemos visualizar as telas que cada endpoint leva:

A imagem mostra uma interface de login com o título "Login" em azul. Abaixo dele, há dois campos de entrada: "Usuário" e "Senha", ambos com bordas arredondadas e fundo branco. Abaixo dos campos, há um botão azul com o texto "Entrar" em branco.

Tela de login – garantir a segurança do sistema



Tela de monitoramento de energia e ambiental



Botões de Configurações e Logout



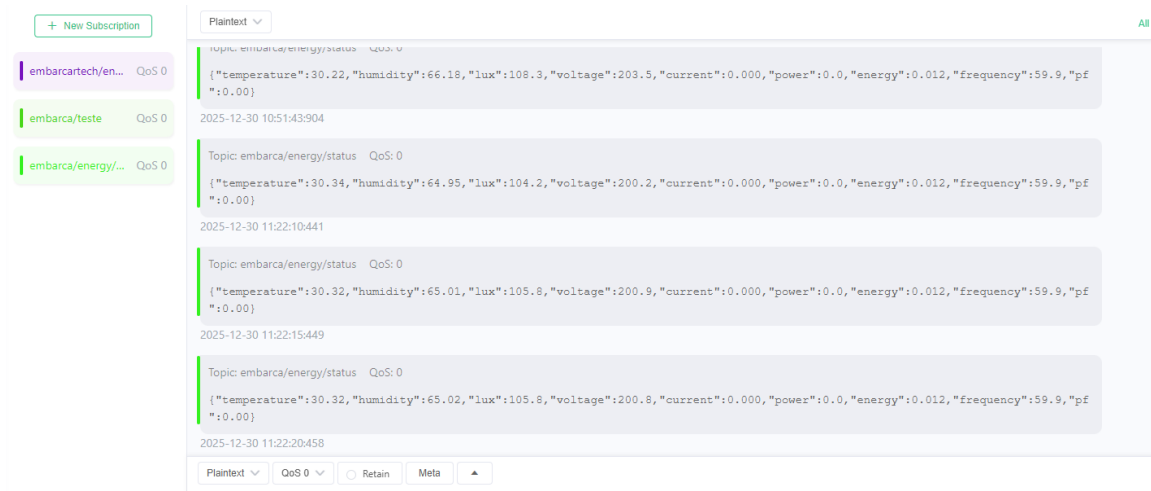
Configuração do RTC – Relógio de tempo Real



Tela de alteração de credenciais de login

5.3 Arquitetura IoT

A camada IoT é responsável pela transmissão dos dados coletados para a nuvem, utilizando o protocolo MQTT. Os dados são publicados periodicamente em tópicos específicos, permitindo sua visualização em dashboards e armazenamento para análise histórica.



6 DESENVOLVIMENTO DO SISTEMA

O desenvolvimento do sistema envolveu a implementação de tarefas independentes no FreeRTOS, cada uma responsável por uma funcionalidade específica, como leitura de sensores, atualização do display e comunicação MQTT. A troca de informações entre tarefas é realizada por meio de filas, evitando acesso concorrente direto aos recursos compartilhados.

O arquivo principal da aplicação é responsável pela inicialização do sistema e criação das tarefas do FreeRTOS. Os módulos de sensores concentram a leitura de variáveis ambientais, enquanto o módulo de energia realiza a comunicação com o medidor PZEM-004T por meio do protocolo Modbus RTU.

O módulo de interface gráfica é responsável pela atualização do display local, apresentando ao usuário os dados coletados em tempo real. Já o módulo de comunicação MQTT gerencia a conexão Wi-Fi, a serialização dos dados e sua publicação no broker MQTT.

As tarefas do FreeRTOS foram projetadas de forma a minimizar dependências diretas entre módulos. Cada tarefa executa uma função específica e se comunica com as demais por meio de filas.

Essa abordagem garante maior robustez ao sistema, permitindo que falhas pontuais em um módulo não comprometam o funcionamento global da aplicação.

```

C energy.c > ...
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  #include "FreeRTOS.h"
5  #include "task.h"
6
7  /* App */
8  #include "app_init.h"
9
10 int main(void)
11 {
12     stdio_init_all();
13
14     //printf("Iniciando Monitorament de energia..\n");
15
16     app_init();
17
18     vTaskStartScheduler();
19
20     while (true) {}
21 }

```

```

40 void app_init(void)
41 {
42     /* ----- BOOT VISUAL RÁPIDO ----- */
43     ST7735_Init();
44     ST7735_FillScreen(ST7735_BLACK);
45     ST7735_SetRotation(1); /* Paisagem: 160x128 */
46     ST7735_DrawString(25, 50, "Inicializando...", Font_7x10, ST7735_WHITE, ST7735_BLACK);
47
48     /* Estado global */
49     system_state_init();
50
51     /* ----- FACTORY RESET ----- */
52     factory_reset_init();
53     factory_reset_task_start();
54
55     /* Eventos da UI */
56     ui_event_group = xEventGroupCreate();
57
58     /* Botão */
59     ui_button_init();
60
61     /* Wi-Fi */
62     wifi_manager_init();
63
64     /* MQTT */
65     mqtt_manager_init();
66
67     /* Tasks */
68     app_tasks_start();
69 }

```

```

16 void app_tasks_start(void)
17 {
18     /* Tasks de aquisição */
19     task_env_sensors_start();
20     task_energy_start();
21     time_manager_start();
22
23     /* Task UI (display) */
24     task_ui_start();
25
26     /* Task lwIP (infraestrutura) */
27     xTaskCreate(
28         vTaskLWIPPoll,
29         "lwip",
30         512,
31         NULL,
32         3,
33         NULL
34     );
35
36     /* Tasks MQTT */
37     xTaskCreate(
38         vTaskMQTTConnection,
39         "mqtt_conn",
40         1024,
41         NULL,
42         2,
43         NULL
44     );
45
46     xTaskCreate(
47         vTaskMQTTPublisher,
48         "mqtt_pub",
49         1024,
50         NULL,
51         2,
52         NULL
53     );
54
55     /* Task de payload MQTT */
56     xTaskCreate(
57         vTaskMQTTPayload,
58         "mqtt_payload",
59         1024,
60         NULL,

```

Os protocolos de comunicação utilizados incluem I2C, UART, SPI e MQTT, garantindo integração eficiente entre os diversos módulos do sistema.

```

1  #ifndef MQTT_MANAGER_H
2  #define MQTT_MANAGER_H
3
4  #include <stdbool.h>
5
6  #ifdef __cplusplus
7  extern "C" {
8  #endif
9
10 /* Tipo de mensagem MQTT */
11 typedef struct {
12     char topic[64];
13     char payload[512];
14 } mqtt_message_t;
15
16 /* API pública */
17 void mqtt_manager_init(void);
18 void mqtt_publish_async(const char *topic, const char *payload);
19
20 void vTaskMQTTConnection(void *pv);
21 void vTaskMQTTPublisher(void *pv);
22
23 bool mqtt_is_connected(void);
24
25 #ifdef __cplusplus
26 }
27 #endif
28
29 #endif /* MQTT_MANAGER_H */
30

```

7 EVIDÊNCIAS DE FUNCIONAMENTO

Como evidências do funcionamento do sistema, foram coletados registros fotográficos da montagem do hardware, capturas do display exibindo dados em tempo real, logs de comunicação MQTT comprovando a publicação dos dados e capturas de tela da plataforma IoT.

Adicionalmente, foi produzido um vídeo demonstrativo com duração entre cinco e sete minutos, apresentando o funcionamento completo do sistema, desde a inicialização até a visualização dos dados na nuvem.

8 CONCLUSÃO

O sistema embarcado desenvolvido atendeu aos objetivos propostos, demonstrando a viabilidade da integração entre hardware, firmware multitarefa e serviços IoT em uma plataforma de baixo custo. A utilização do FreeRTOS proporcionou maior organização e previsibilidade ao firmware, enquanto a comunicação MQTT garantiu a transmissão dos dados.

O projeto apresenta potencial de expansão, podendo ser adaptado para novas aplicações e integrações futuras.

9 REFERÊNCIAS

FREERTOS. FreeRTOS Reference Manual. Disponível em: <<https://www.freertos.org>>. Acesso em: 1 jan. 2026.

MQTT. MQTT Version 3.1.1 Specification. Disponível em: <<https://mqtt.org>>. Acesso em: 1 jan. 2026.

RASPBERRY PI FOUNDATION. Raspberry Pi Pico W Documentation. Disponível em: <<https://www.raspberrypi.com>>. Acesso em: 1 jan. 2026.

ALMEIDA, Rodrigo Maximiano Antunes de. Programação de Sistemas Embarcados: desenvolvimento de software para microcontroladores em linguagem C / Rodrigo Maximiano Antunes, Carlos Henrique Valéric Moraes, Thatyana de Farias Piola Seraphim. – 1ª Edição – Rio de Janeiro: Elsevier, 2016.

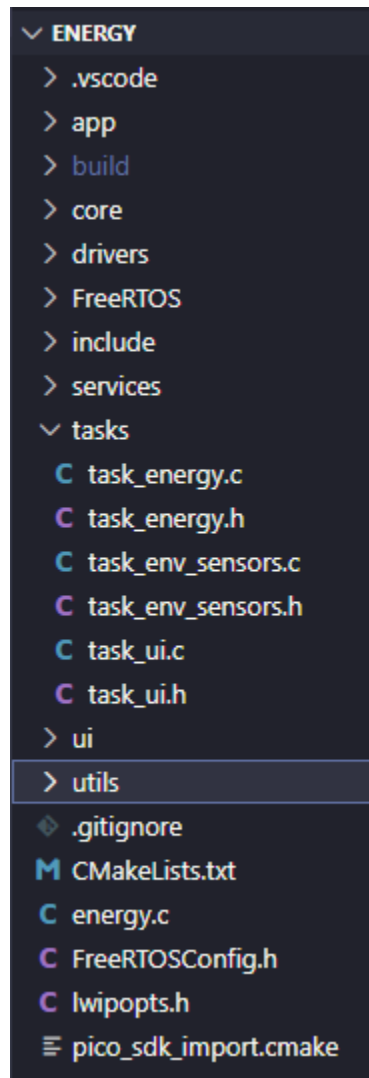
DENARDIN, Gustavo Weber. Sistemas Operacionais de Tempo Real e Suas Aplicações em Sistemas Embarcados / Gustavo Weber Danardin, Carlos Henrique Barriquello. São Paulo: Blucher 2019.

WHITE, Elecia. Construindo Sistemas Embarcados – padrões de Projeto para um Ótimo Software. 2ª Edição – O'REILLY – Novatec 2024.

10 APÊNDICES

Este apêndice apresenta uma análise detalhada da organização do firmware desenvolvido, bem como a estrutura de diretórios do projeto, com o objetivo de facilitar o entendimento da arquitetura interna do sistema.

O firmware foi desenvolvido em linguagem C, utilizando o SDK oficial da Raspberry Pi Pico W e o sistema operacional de tempo real FreeRTOS. A organização do projeto segue boas práticas de engenharia de software embarcado, com separação clara entre inicialização do sistema, tarefas, drivers e bibliotecas.



O sistema embarcado desenvolvido incorpora uma interface web para configuração e monitoramento local, acessível por meio de navegador padrão em dispositivos conectados à mesma rede. Essa interface foi implementada diretamente no firmware, sem dependência de servidores externos, utilizando a pilha de rede lwIP em modo RAW TCP.

A interface web é composta por páginas HTML responsivas, incluindo tela de login, painel de monitoramento (dashboard) e páginas de configuração. O painel principal apresenta, em tempo real, dados de energia elétrica, como tensão, corrente, potência, energia consumida, frequência e fator de potência, além de variáveis ambientais, como temperatura, umidade e luminosidade.

O acesso ao servidor web é protegido por um mecanismo de autenticação baseado em credenciais de usuário e senha. Após a autenticação bem-sucedida, o sistema cria uma sessão válida, controlada por tempo de inatividade. Caso não haja interação do usuário dentro de um período previamente definido, a sessão é automaticamente invalidada, exigindo novo login.

Esse mecanismo reduz riscos associados a acessos não autorizados, especialmente em ambientes onde o dispositivo permanece continuamente conectado à rede.

Visando aumentar a segurança do sistema, as senhas de acesso não são armazenadas em texto plano. Durante o processo de configuração das credenciais, a senha fornecida pelo usuário é processada por um algoritmo criptográfico de hash SHA-256, gerando um valor de 256 bits.

Apenas o hash resultante da senha é armazenado de forma persistente na memória EEPROM do módulo RTC. Dessa forma, mesmo em caso de acesso indevido à memória, a senha original não pode ser diretamente recuperada, atendendo a boas práticas de segurança da informação.

A utilização da memória EEPROM integrada ao módulo RTC para o armazenamento das credenciais proporciona maior confiabilidade ao sistema, uma vez que as informações permanecem preservadas mesmo em casos de desligamento total da alimentação principal.

Além das credenciais de autenticação, o módulo RTC também é utilizado para manter a data e hora do sistema, permitindo registros temporais consistentes e possibilitando futuras expansões, como geração de logs com carimbo de tempo.

O servidor web disponibiliza endpoints HTTP específicos para fornecimento de dados em formato JSON, os quais são consumidos pela própria interface web por meio de requisições assíncronas. Essa abordagem permite a atualização periódica das informações exibidas no dashboard sem a necessidade de recarregar a página.

Os dados apresentados são obtidos diretamente da estrutura de estado global do sistema, garantindo que as informações exibidas ao usuário reflitam fielmente as medições mais recentes realizadas pelos sensores e pelo módulo de energia.